

Contents

Arcade Architecture Manual	1
Startup Flow	1
Runtime Linking	1
How Procedures Are Linked	1
1. Core Layer	1
2. Display Layer	2
3. Game Layer	2
Switching Rules	2
Implementation Advice	2

Arcade Architecture Manual

Startup Flow

1. `main.cpp` checks that a graphical library path is provided on the command line.
2. `arcade::Core::run()` loads the initial graphical library and stores the list of available `.so` files from `./lib`.
3. The selected display module is initialized with the window title and size.
4. The core asks the player name through the active display.
5. The menu is shown, then the player can start a game, change renderer, inspect scores, or quit.

Runtime Linking

The runtime contract is defined by `DLLoader<T>` in `src/DLLoader.hpp`. Each library must export:

- `myEntryPoint()` to create the module instance.
- `getLibType()` to tell whether the module is graphical or a game.
- `getLibName()` to provide the label shown in the menus.

The core validates the type before using the module. A graphical library is rejected if `getLibType()` does not return `GRAPHICAL`, and a game library is rejected if it does not return `GAME`.

How Procedures Are Linked

The program is split into three layers.

1. Core Layer

`Core` manages global state, library switching, user name input, score display, and the current active game and graphical modules.

It also decides when to reload a library:

- `changeNewDisplay()` cycles through the available graphical libraries.
- `changeNewGame()` cycles through the available game libraries.
- `loadGraphic()` and `loadGame()` perform the actual dynamic loading.

2. Display Layer

Any class that implements `IDisplayModule` is responsible for:

- creating the window,
- creating `IRect` objects,
- mapping backend events to `arcade::EEvent`,
- rendering the scene,
- managing optional audio support.

The core never talks to the backend directly. It only manipulates the common interface and the returned rectangles.

3. Game Layer

Any class that implements `IGameModule` owns the game state and updates the scene through the display interface.

The game can request a mode change by setting internal flags that are exposed through:

- `changeDisplay()`
- `changeGame()`
- `exit()`

The core reads those flags after every `run()` call and reacts accordingly.

Switching Rules

- Pressing `TAB` in the menu or in a running game asks the core to cycle to the next compatible graphical library.
- The game can request a game change through its own internal control flow.
- When the user quits, the core closes the active libraries and ends the process.

Implementation Advice

- Keep all backend-specific code inside the graphical libraries.
- Keep all gameplay rules inside the game libraries.
- Use the shared `Texture`, `Size`, `Position`, `Bounds`, and `EEvent` types everywhere at the interface boundary.
- Prefer small, single-purpose methods so the extension code remains easy to audit and reuse.